

TECH DEEP DIVE · DOCKER & LOKALE KI

KI Agent mit Ollama und iGPU

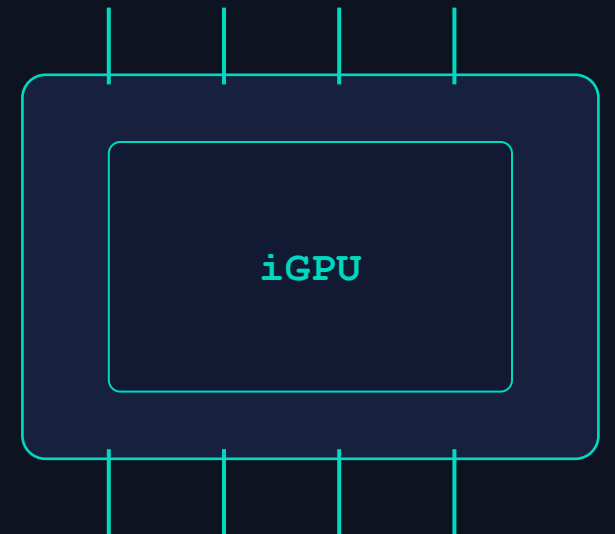
Vom Grundverständnis zur optimierten iGPU-Nutzung

Grundlagen · Hardware · Optimierung · Messergebnisse

Dokumentversion 28.09.2026

Projekt von <https://webwork.pro>

Dateien <https://github.com/Ralfhgw/igpu-performance-tuning.git>



BLOCK 1 VON 4

Grundlagen

- 1 Einführung: Lokale KI auf dem Mini-PC
- 2 Wie viel Speicher benötigt ein KI-Modell?
- 3 Modelle im Speicher halten: Sleep Time / Keep Alive
- 4 Token statt Wörter: Wie KI Text verarbeitet
- 5 Verfügbare Modelle: Ein Überblick über ollama.com/library
- 6 Ollama REST-API: Die wichtigsten Endpunkte
- 7 Tool Calling: KI ruft eigene Werkzeuge auf

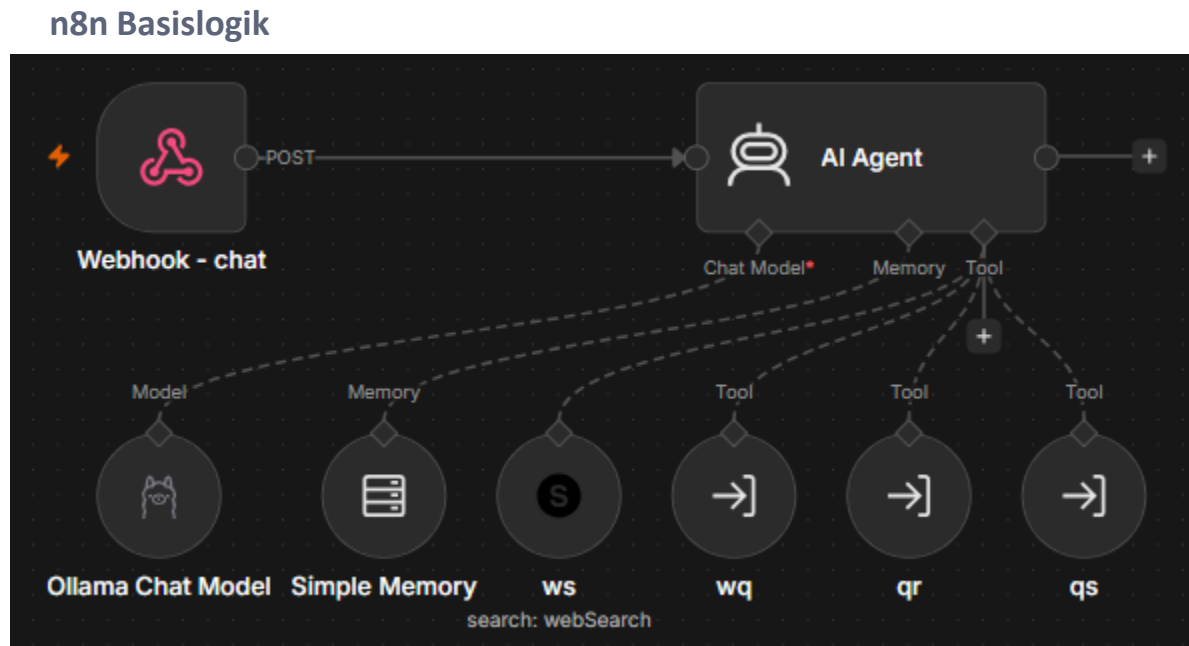


NiPoGi AM06 PRO Mini PC mit Ryzen 7
32 GB RAM und 512 GB SSD Festplatte

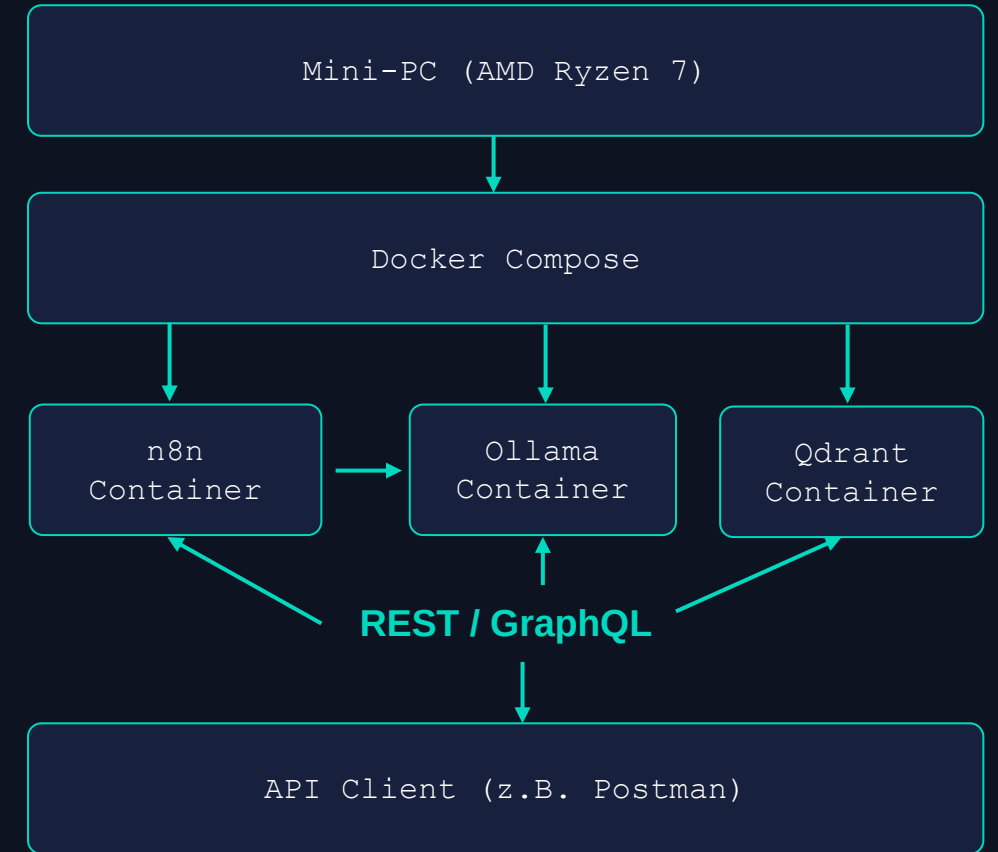
Lokale KI auf dem Mini-PC

Was soll untersucht werden?

Wie effizient lassen sich Sprachmodelle auf einem kleinen Rechner mit Docker betreiben?



Der Testaufbau



Versionsabfrage:

```
$ docker exec -it n8n_container n8n --version → 2.40.7  
$ curl -s http://localhost:6333/ | jq -r '.version' → 1.19.1  
$ docker exec -it ollama_container ollama --version → 0.34.2
```

Wie viel Speicher benötigt ein KI-Modell?

Ein geladenes Modell reserviert dauerhaft Arbeitsspeicher – ein Vorher-Nachher-Test mit llama3.1:8b macht das sichtbar.

```
$ docker compose up -d ollama
$ docker exec -it ollama_container ollama list
$ free -h
$ docker exec -it ollama_container \
  ollama run llama3.1:8b \
    "Beschreibe in einem Wort wer du bist"

$ docker exec -it ollama_container ollama ps
$ free -h
$ docker compose stop ollama

# Ergebnis: RAM-Verbrauch steigt sichtbar
# um die Modellgröße, sobald geladen wird
```

4,9 GB

RAM-Bedarf von llama3.1:8b im geladenen Zustand

Und beim Entladen?

Wird der Container gestoppt, gibt das System den belegten Speicher vollständig wieder frei.

Modelle im Speicher halten: Sleep-Time

„keep_alive“ bestimmt, wie lange ein Modell nach der letzten Anfrage geladen bleibt, bevor Ollama es automatisch entlädt.

s / m / h

Sekunden, Minuten oder Stunden bis zum Entladen

-1

Unendlich lange – bis Ollama beendet wird

0

Wird sofort nach der Antwort gelöscht

Aktivieren per Docker Compose:

```
docker-compose.yml:  
  OLLAMA_KEEP_ALIVE=-1
```



```
$ docker exec -it ollama_container ollama ps  
NAME      ID          SIZE  PROC  CONTEXT  UNTIL  
llama3.1:8b 46e0c10c 5.6GB 100% CPU 4096   4 min  
  
$ docker compose up -d --force-recreate ollama  
  
$ docker exec -it ollama_container ollama ps  
NAME      ID          SIZE  PROC  CONTEXT  UNTIL  
llama3.1:8b 46e0c10c 5.6GB 100% CPU 4096   Forever  
  
# Unterschied:  
# 'X min' -> zeitweises Halten (Timeout)  
# 'Forever' -> dauerhaft im Speicher geladen
```

Token statt Wörter: Wie KI Text verarbeitet

Ein Token ist die kleinste Texteinheit, mit der ein Sprachmodell rechnet – oft ein Wortteil statt eines ganzen Wortes. Die Anzahl der Token hängt von der verwendeten Sprache und der Textstruktur ab und kann vorab mit einem Tokenizer bestimmt werden (z.B. llm-tokenizer.com).

DEUTSCH

Datenschutzbestimmungen

Daten + schutz + best + imm + ung + en

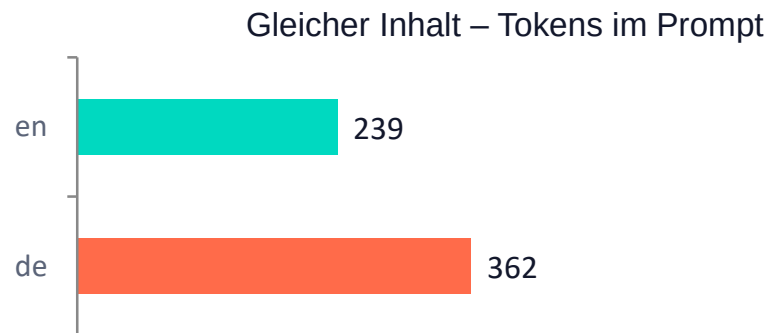
≈ 5–6 Tokens für ein Wort

ENGLISCH

Data protection regulations

Data + protection + regulations

= genau 3 Tokens (1 pro Wort)



Token Berechnung in Ollama

```
$ jq -n --rawfile t example-deutsch.txt \  
'{model:"llama3.1:8b",prompt:$t,stream:false,options:{num_predict:1}}' | \  
curl -s http://localhost:11434/api/generate -d @- | \  
jq '{prompt_eval_count, eval_count}'  
  
"prompt_eval_count": 319,  
"eval_count": 1
```

Verfügbare Modelle: Ein Überblick über ollama.com/library

Ollama kann hunderte offene Modelle laden – z. B. mit `docker exec -it ollama_container ollama pull <modell:tag>`. Diese sieben Familien decken die meisten Alltags- und Spezialfälle ab. Verfügbare Modelle in <https://ollama.com/library>

Modell-Familie	Beispiel-Tag	Anbieter / Herkunft	Stärken
Llama 3.1 / 3.2 / 3.3	llama3.2:3b	Meta AI (USA)	Weltweiter Open-Source-Standard – stark in Textgenerierung, logischem Denken und Alltagskonversation.
Gemma 4	gemma4:12b	Google (USA)	Exzellent im Befolgen komplexer Instruktionen und bei Mathe-Aufgaben – Vision & Tool-Calling ab Werk.
Mistral Nemo / Large	mistral-nemo:12b sauerkrautlm-nemo-12b-instruct	Mistral AI (FR) & NVIDIA	Europäisches Spitzenmodell – besonders stark in europäischen Sprachen (exzellentes Deutsch) und Argumentation.
Qwen 3.6 / Qwen3	qwen3:32b	Alibaba (China)	Schlägt in Benchmarks regelmäßig westliche Modelle – hervorragend für allgemeine und mehrsprachige Aufgaben.
DeepSeek R1	deepseek-r1:70b	DeepSeek (China)	Reasoning-Modell mit interner Denkkette (Open-Source-Antwort auf o1/o3 von OpenAI) – extrem stark in Mathe, Logik und Programmierung.
Qwen3 Coder	qwen3-coder:30b	Alibaba (China)	Führendes lokales Modell für Programmierassistenz – Code-Generierung, Bugfixing, große Codebases.
Phi-4	phi4:14b	Microsoft (USA)	Kompakt und hochoptimiert – trotz geringerer Parameterzahl präzise bei wissenschaftlichen und logischen Aufgaben.

Ollama REST-API: Die wichtigsten Endpunkte

Ollama stellt zwei Endpunkt-Familien bereit: native Endpunkte unter `/api/*` und OpenAI-kompatible Endpunkte unter `/v1/*` – für Tools, die eigentlich für die ChatGPT-API geschrieben wurden. Aufruf z.B. mit:

```
curl -s "http://localhost:11434/api/show" -H 'Content-Type: application/json' -d '{"model\":\"myaniu/qwen2.5-1m:14b\"}'
```

TEXT- & INFERENZ-ENDPUNKTE

POST /api/chat	Strukturierte Konversation über ein messages-Array (System-/User-/Assistant-Rollen).
POST /api/embed	Erzeugt Vektor-Embeddings aus Text – Basis für eigene Wissensdatenbanken (RAG).

MODELLVERWALTUNG

GET /api/tags	Listet alle lokal installierten, einsatzbereiten Modelle auf.
POST /api/show	Zeigt System-Prompt, Lizenz und Architektur eines installierten Modells.
GET /api/ps	Zeigt, welche Modelle gerade im RAM/VRAM geladen sind.
POST /api/pull	Lädt ein Modell aus der Ollama-Bibliothek im Hintergrund herunter.
POST /api/push	Lädt ein selbst erstelltes Modell (Modelfile) in die Bibliothek hoch.
POST /api/create	Erstellt ein neues Modell aus einem lokalen Modelfile.
POST /api/copy	Dupliziert ein lokales Modell unter neuem Namen.
DELETE /api/delete	Löscht ein lokales Modell und gibt Festplattenplatz frei.

OPENAI-KOMPATIBILITÄT (/v1/*)

POST /v1/chat/completions	Verhält sich exakt wie die OpenAI-Chat-API – bestehende Tools laufen ohne Codeänderung.
POST /v1/embeddings	OpenAI-kompatibler Embedding-Endpoint, gleiche Request-/Response-Struktur.
GET /v1/models	Listet die lokalen Modelle im OpenAI-Format auf.

Tool Calling: KI ruft eigene Werkzeuge auf

Mit „tools“ im Request bekommt das Modell Funktionen angeboten. Ob es sie wirklich nutzt, statt sie nur zu simulieren, zeigt der Vergleich zweier Antworten. Eine einfache Anwendung für eigene Tests - **tool_call.py**)

Request an Ollama

```
curl http://localhost:11434/api/chat -d '{
  "model": "cyberwald/sauerkrautlm-nemo-12b-instruct:latest",
  "messages": [{"role": "user", "content": "Lies https://www.anthropic.com und sag mir was drauf steht."}],
  "tools": [{
    "type": "function",
    "function": {
      "name": "Seite_lesen",
      "description": "Liest den Inhalt einer Webseite",
      "parameters": {"type": "object", "properties": {"url": {"type": "string"}}, "required": ["url"]}
    }
  ]
},
  "stream": false
}'
```

Response von Ollama

Ohne Tool Call

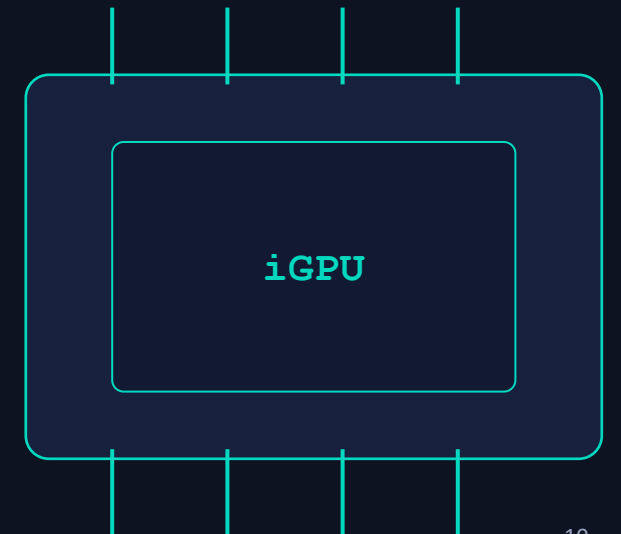
```
... "message": {"role": "assistant", "content": "Ich werde die Webseite lesen..."}
```

Mit Tool Call

```
... "message": {"role": "assistant", "content": "", "tool_calls": [{"id": "call_xs2usa3w", "function": {"index": 0, "name": "Seite_lesen", "arguments": {"url": "https://www.anthropic.com"}}}]}
```

Verständnis der Verarbeitung

- 1 Kontextfenster: Der Arbeitsspeicher der KI
- 2 Praxistest: Ein Geheimwort am Ende eines langen Textes
- 3 CPU, RAM, VRAM, iGPU, NPU – die Hardware hinter der KI
- 4 NVIDIA-GPU vs. Apple Silicon vs. AMD-iGPU
- 5 Speicherbandbreite als Flaschenhals



Kontextfenster: Der Arbeitsspeicher der KI

Die „Context Length“ bestimmt, wie viele Token das Modell gleichzeitig im Gedächtnis behalten kann. Ollama legt die Kontextgröße basierend auf dem verfügbaren VRAM fest. Das Modell und der dafür reservierte KV-Cache müssen vollständig in den VRAM passen, um langsame CPU-Auslagerungen zu verhindern.

< 24 GB VRAM

4k Kontext

Ollama-Standard für Consumer-Karten.
Ausreichend für kurze Chats / ca. 3.000
Wörter

24–48 GB VRAM

32k Kontext

Mehr Spielraum: Ideal für Datenanalysen,
RAG-Systeme und längere Programmier-
Kontexte

> 48 GB VRAM

256k Kontext

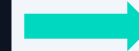
Enterprise-Niveau: Ermöglicht die
Verarbeitung ganzer Dokumente und
Bücher auf einmal



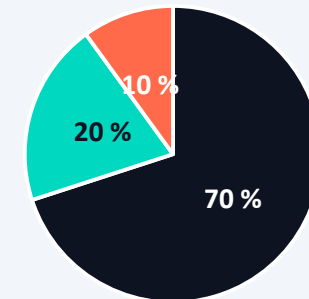
Faustregel: Ein Token entspricht im Durchschnitt etwa 4 Zeichen oder drei Viertel eines Wortes.

GESAMT-VRAM · schematische Verteilung

Modell auf
Festplatte



wird beim Start in den VRAM geladen



- **Modell-Gewichte** – fest definiert durch das Modell
- **KV-Cache (Kontext)** – wächst mit num_ctx! Hier liegen System-Prompt, Chat-Historie (3 Frage-Antwort-Paare), Tool-Ergebnisse und die aktuelle Generierung.
- **Zwischenspeicher (Activation Memory)** – „Scratchpad“ der Berechnung: bläht sich besonders in der Prefill-Phase kurz auf und wird danach sofort wieder freigegeben.



Ist die Summe aus Modell-Gewichten, KV-Cache und Zwischenspeicher größer als der VRAM → sofortiger Performance-Einbruch (CPU-Fallback)!

Praxistest: Ein Geheimwort am Ende eines langen Textes

Ein Testtext mit einem versteckten Codewort ganz am Ende prüft, ob die KI wirklich den kompletten Kontext verarbeitet.



```
# Schritt 1: kurzer Text (60 Zeilen) + Codewort
$ yes "This is a long text for the AI test" | head -n 60 > ki-test.txt
$ echo "GEHEIMWORT: ANANASSAFT" >> ki-test.txt
$ docker exec -it ollama_container ollama run llama3.1:8b "Lies den folgenden Text
aufmerksam durch. Ganz am Ende steht ein geheimes Codewort nach dem Wort 'GEHEIMWORT:'.
Nenne mir nur dieses eine Codewort und absolut nichts anderes! Hier ist der Text: $(cat
ki-test.txt)" --verbose
ANANASSAFT

# Schritt 2: gleicher Test mit 600 Zeilen
$ yes "...Testtext..." | head -n 600 > ki-test.txt
$ echo "GEHEIMWORT: ANANASSAFT" >> ki-test.txt
$ ollama run llama3.2:latest "..." --verbose
# Standardkontext (4k) reicht nicht mehr zuverlässig

# Schritt 3: Kontext auf 16k erhöhen
docker-compose.yml:
  OLLAMA_CONTEXT_LENGTH=16384
$ docker compose up -d --force-recreate ollama
# -> Codewort wird wieder zuverlässig gefunden
```

Was zeigt der Test?

Bei kurzen Texten liefert schon der Standardkontext (4k) zuverlässig das richtige Codewort.

Wächst der Text über die eingestellte Kontextgröße hinaus, muss diese aktiv erhöht werden – sonst gehen Informationen am Anfang des Textes verloren.

CPU, RAM, VRAM, iGPU, NPU – die Hardware hinter der KI



Wenige, vielseitige Kerne – rechnet sequenziell und ist für die riesigen Matrixmultiplikationen einer KI vergleichsweise langsam.



Tausende einfache, parallele Rechenkerne – ideal für genau die Matrixoperationen, aus denen ein Sprachmodell besteht.



Hochspezialisierte, fest verdrahtete Rechenschaltkreise – führt Matrixoperationen extrem parallel und direkt auf der Hardware aus.

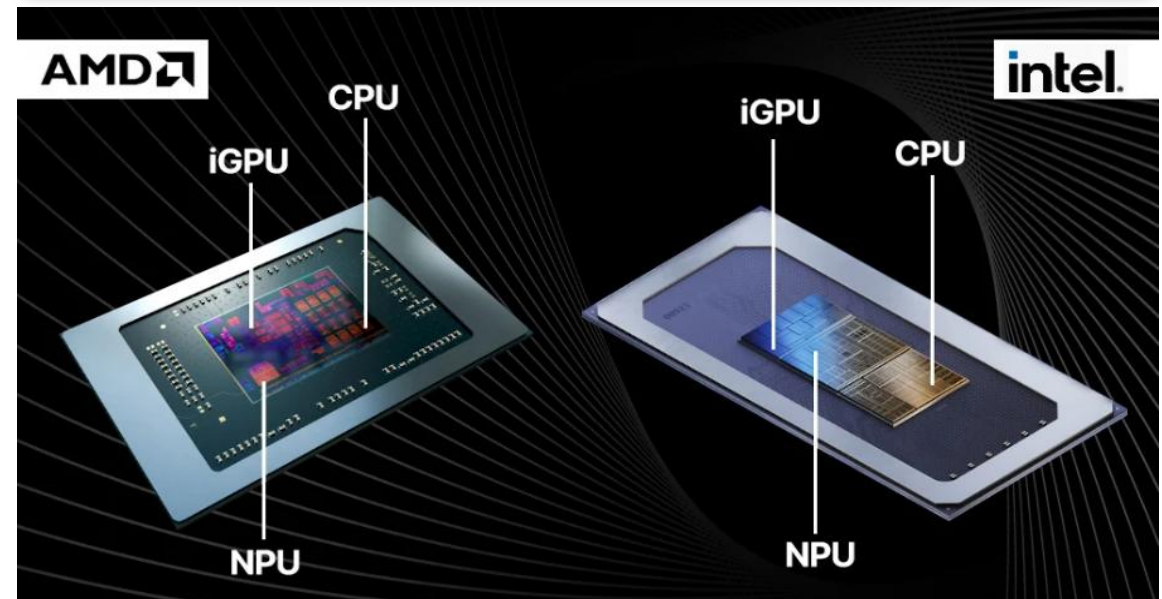
Der iGPU-Sonderfall: AMD-CPU's, die auf G, U, HS oder H enden (sowie fast alle Notebook-CPU's), haben eine integrierte GPU. Bei Intel gilt das für fast alle Modelle – außer bei einem „F“ im Namen (z. B. i5-12400F).



```
$ lscpu | grep "Model name"
Model name: AMD Ryzen 7 5825U with Radeon Graphics

$ sudo dmesg | grep -i -E "vram|gtt"
VRAM: 512M ready | GTT: 15755M (≈15,4 GB) ready

# GTT = Graphics Translation Table: RAM-Anteil, den
# sich die iGPU zusätzlich zum kleinen VRAM leihen darf
```



NVIDIA-GPU vs. Apple Silicon vs. AMD-iGPU

	NVIDIA dGPU	Apple Silicon	AMD-iGPU
Speicher	Getrennt: RAM + VRAM	Unified Memory auf Basis der LPDDR5X, direkt auf dem Prozessor-Chip (SoC – System on a Chip) verlötet	Nutzt GTT-Bereich im System-RAM
Datentransport	Kopieren über PCIe	Direkter Zugriff, kein Kopieren	Direkter Zugriff über ROCm / Vulkan
Flaschenhals	VRAM-Größe begrenzt Modell, PCIe-Nadelöhr bei RAM-Auslagerung	Freier RAM wird dynamisch geteilt	Geteilte RAM-Bandbreite
Bandbreite	300+ GB/s (GDDR6)	M3, M4, M6 - 100 bis 120 GB/s M3 Pro, M4 Pro, M5 Pro – 150 bis 307 GB/s M3 Max, M4 Max, M5 Max – 400 bis 614 GB/s M3 Ultra, M5 Ultra – 800 bis 1,2 TB/s	≈ 50 – 100+ GB/s (DDR4 bis LPDDR5x)
Energie	Hoher Verbrauch, sehr stark	Extrem effizient	Effizient, aber begrenzt

Speicherbandbreite bei dedizierten Grafikkarten (dGPU)



Speichertechnologien im Vergleich

RAM-Technologien unterscheiden sich grundlegend in Architektur, Einsatzzweck und Speicherbandbreite – genau daran liegt es, wenn die iGPU bei sehr langem Kontext (z. B. 1 Mio. Tokens) einknickt.

Speichertyp	Primärer Einsatzzweck	Bandbreite	Hauptmerkmal
DDR4 / DDR5	Arbeitsspeicher (Desktop-PC, Server, Mini-PC)	ca. 25–90 GB/s	Steckbar (DIMM/SO-DIMM). Optimiert auf niedrige Latenz statt maximaler Bandbreite.
LPDDR5 / 5x	Smartphones, Tablets, dünne Laptops	ca. 50–120 GB/s	"Low Power" – fest verlötet, stromsparend, etwas höhere Latenz als DDR5.
GDDR6 / 6X	Grafikkarten (VRAM), Spielekonsolen	ca. 500–1.000+ GB/s	"Graphics" – massive Bandbreite für Texturen & KI-Gewichte. Stromhungrig & heiß.
HBM (3 / 4)	High-End-KI-Beschleuniger (NVIDIA H100, AMD MI300)	ca. 2.000–4.900+ GB/s	Vertikal gestapelt, direkt am Chip. Leistungsmaximum – extrem teuer.

Bauformen: Wo sitzt der Speicher?

DIMM Desktop-RAM Klassischer, langer Riegel für Desktop-PCs. Leicht austausch- und aufrüstbar.	SO-DIMM Laptop-RAM Halbe Länge, in Laptops & Mini-PCs. Aufrüstbar (24–64 GB), teilt sich aber die magere RAM-Bandbreite.	LPDDR Fest verlötet / On-Board Chips direkt auf der Platine neben der CPU. Kürzere Signalwege → höhere Taktraten (bis 8533 MHz), spürbarer iGPU-Boost.	CAMM2 Der neue Standard Flaches, schraubbares Modul. Vereint LPDDR5x-Tempo & Effizienz mit Austauschbarkeit.
--	--	--	---

Speicherbandbreite berechnen: Der Dual-Channel-Check

Ein Blick in dmidecode verrät nicht nur, wie viel RAM verbaut ist, sondern auch, wie schnell er tatsächlich angebunden ist.

```
$ sudo dmidecode --type 17 | \
    grep -E "Type:|Speed:|Size:|Locator:"

# DIMM 0 - Bank Locator: P0 CHANNEL A
Size: 16 GB      Locator: DIMM 0
Type: DDR4      Speed: 2667 MT/s

# DIMM 0 - Bank Locator: P0 CHANNEL B
Size: 16 GB      Locator: DIMM 0
Type: DDR4      Speed: 2667 MT/s
```

42,67 GB/s

Speicherbandbreite (DDR4-2667, Dual-Channel)

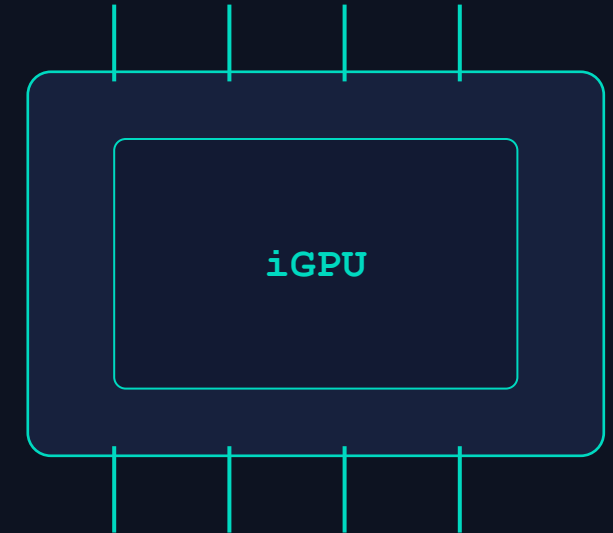
Was die Ausgabe verrät

Die Werte zeigen die optimale Aufteilung deines Arbeitsspeichers für maximale Performance unter Linux. Daraus ergeben sich drei Kernaspekte:

- 32 GB physischer RAM (30 GiB für das OS): Es sind zwei Riegel mit jeweils 16 GB verbaut. Davon sind im BIOS lediglich 512 MB fest als VRAM reserviert. Der Kernel sieht somit fast den gesamten Speicher (ca. 30 GiB).
- Dynamischer Grafikspeicher (GTT): Über den Kernel-Parameter `amdgpu.gttsize=26000` stehen der iGPU bei Bedarf bis zu 26.000 MB des normalen RAMs dynamisch zur Verfügung. Moderne Grafik-Schnittstellen wie Vulkan (oder ROCm für Ollama) können diesen riesigen GTT-Bereich nahtlos ansprechen, ohne dass der Speicher im BIOS blockiert werden muss.
- Optimaler Dual-Channel-Modus: Da sowohl CHANNEL A als auch CHANNEL B belegt sind, läuft der RAM im Dual-Channel-Betrieb. Das verdoppelt die Datenautobahn zum Prozessor im Vergleich zu einem einzelnen Speicherriegel (Single-Channel).
- DDR4 mit 2667 MT/s: Dein Arbeitsspeicher führt 2.667.000.000 Datenübertragungen pro Sekunde aus.

$$\frac{2667 \text{ MT/s} \times 64 \text{ Bit Busbreite} \times 2 \text{ Kanäle (Dual-Channel)}}{8 \text{ Bit pro Byte}} = 42,67 \text{ GB/s}$$

Optimierung des Mini-PCs



- 1 Vulkan: Die iGPU für lokale KI nutzen
- 2 KI-Geschwindigkeit messen
- 3 Konfiguration: Wer setzt sich durch ?
- 4 docker-compose.yml: Ollama-Service
- 5 docker-compose.yml: Berechtigungen & Kern-Variablen
- 6 docker-compose.yml: Weitere Variablen
- 7 Modelfile: Basismodell, System-Rolle & Prompt-Format
- 8 Vulkan: Die iGPU für lokale KI nutzen
- 9 KI-Geschwindigkeit messen
- 10 Modelfile-Konfiguration: Wer setzt sich durch ?
- 11 docker-compose.yml: Ollama-Service
- 12 docker-compose.yml: Berechtigungen & Kern-Variablen
- 13 docker-compose.yml: Weitere Variablen
- 14 Modelfile: Basismodell, System-Rolle & Prompt-Format

Vulkan: die Brücke zur iGPU

1

Standard für performante 2D-/3D-Grafik – steuert im Hintergrund Millionen Geräte an

2

Erkennt den GTT-Bereich und lädt das KI-Modell in den geteilten RAM

3

Aktiviert die Grafikerne der iGPU – genau deshalb ist Vulkan bei AMD-iGPUs so interessant

Ohne Vulkan sieht Ollama zwar die Hardware, nutzt aber nur die CPU – der Großteil der Rechenleistung der iGPU bleibt ungenutzt.

Docker braucht Zugriff auf:

`/dev/dri`

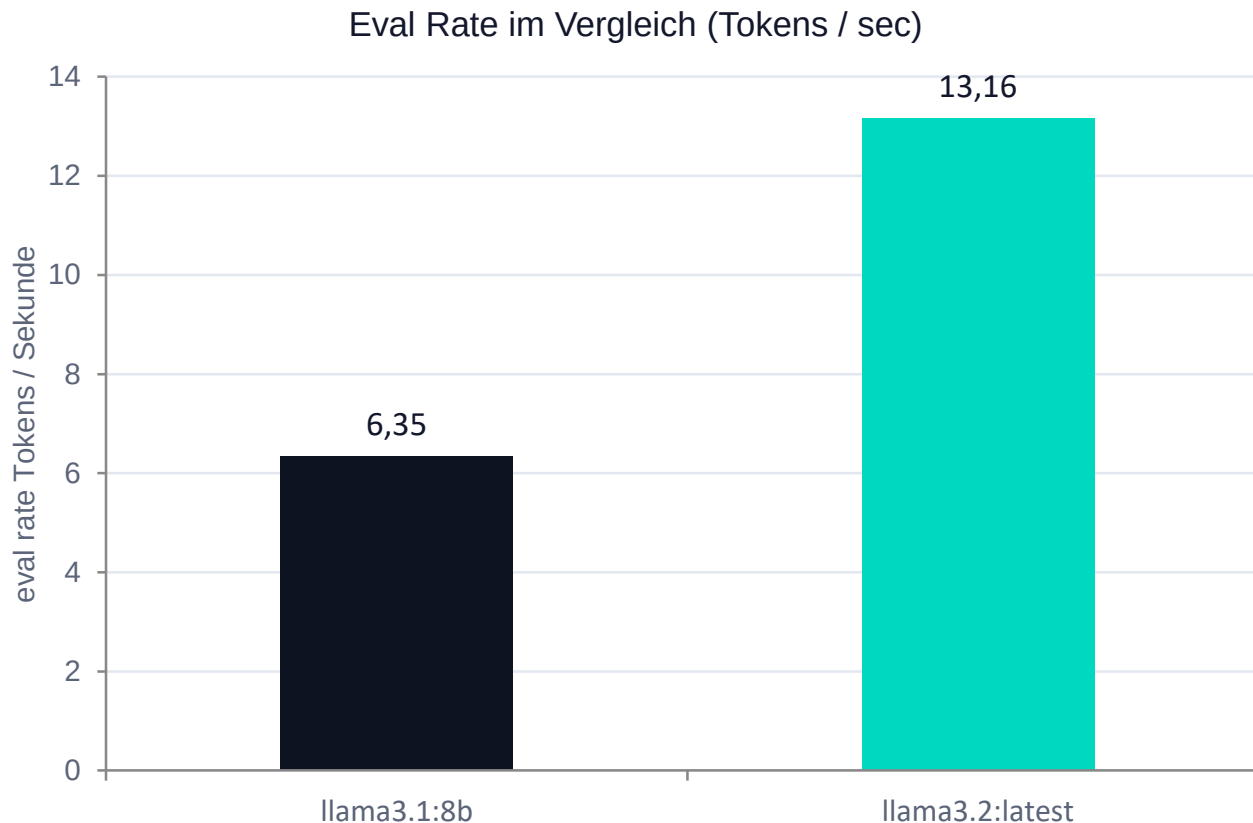
Öffnet dem Container die Tür zur Grafikeinheit (Vulkan-Zugriff).

`/dev/kfd`

AMDs Spezialtür für Compute-Aufgaben – direkte Kommunikation mit dem Prozessorkern für Berechnungen.

KI-Geschwindigkeit messen

Die Anzahl der Parameter in Llama3.1:8b ist hoch im Vergleich zu Llama3.2 und sollte verwendet werden für Komplexe, mehrsprachige Übersetzungen (Nuancen), Tiefes Fach- und Faktenwissen (Deep Knowledge), Programmierung und Code-Generierung und Komplexe Logik und mehrstufige Workflows (Reasoning)



~2x

schneller mit llama3.2:latest
im direkten Codewort-Test

```
$ docker exec -it ollama_container ollama run llama3.1:8b  
"Lies den folgenden Text aufmerksam durch. Ganz am Ende steht  
ein geheimes Codewort nach dem Wort 'GEHEIMWORT:'. Nenne mir  
NUR dieses eine Codewort und absolut nichts anderes! Hier ist  
der Text: $(cat ki-test.txt)" --verbose
```

prompt eval rate:

zeigt, wie schnell der Eingabetext eingelesen wird.

eval rate:

Die Generierungsgeschwindigkeit (Eval Rate) zeigt,
wie viele Tokens das Modell pro Sekunde ausgibt.

Konfiguration: Wer setzt sich durch?

REQUEST > MODELFILE > COMPOSE

Der Wert in der API-Anfrage sticht das Modelfile immer aus. Das Modelfile dient als Fallback – hier hinterlegst du die optimalen Werte als globale Standardwerte.

Wo wird der Wert tatsächlich gesetzt?

n8n Node Einstellungen	Wird zur Laufzeit jeder Anfrage mitgeschickt – sticht IMMER alle anderen Werte aus	Höchste Prio
Modelfile	Der Standardwert des Modells – greift, wenn die Anfrage selbst nichts vorgibt	Fallback
docker-compose.yml	Der globale Server-Deckel – harte Obergrenze für alle Modelle im Container	Deckel
Praxis-Regel	Optimale Werte als globalen Standard im Modelfile hinterlegen, Ausnahmen gezielt per n8n-Node überschreiben	Empfehlung

docker-compose.yml: Ollama-Service

```
image: ollama/ollama:latest
```

Der Container läuft als eigener Service – Port, Volumes und Geräte-Zugriffe werden hier zentral für alle Modelle festgelegt.

Die wichtigsten Bausteine des Service-Blocks

```
ports: "11434:11434"
```

Veröffentlicht die Ollama-API auf dem Standard-Port des Hosts

Pflicht

```
volumes: ./ollama_data:/root/.ollama
```

Speichert Modelle & Modelfiles dauerhaft außerhalb des Containers

Pflicht

```
devices: /dev/dri, /dev/kfd
```

Gibt der iGPU direkten Hardware-Zugriff für Vulkan-Beschleunigung

Pflicht

```
group_add: "44", "993"
```

Fügt den Container zu den render-/video-Gruppen des Hosts hinzu

Wichtig

```
restart: unless-stopped
```

Startet den Container nach Absturz oder Host-Neustart automatisch neu

Empfohlen

docker-compose.yml: Berechtigungen & Kern-Variablen

Neben Geräten und Gruppen-Berechtigungen sorgen diese Einstellungen dafür, dass Ollama die iGPU überhaupt zuverlässig nutzen kann.

DOCKER-COMPOSE-EBENE

```
ipc: host
```

Container teilt den IPC-Namespaces mit dem Host – nötig, damit GPU-Treiber Shared Memory zwischen Prozessen nutzen können

```
ulimits:  
  memlock:  
    soft: -1  
    hard: -1
```

Hebt das Limit für gesperrten (nicht auslagerbaren) Speicher auf, damit der Treiber genug GPU-Speicher reservieren kann

OLLAMA-UMGEBUNGSVARIABLEN

```
OLLAMA_VULKAN=1
```

Aktiviert das Vulkan-Backend, über die Ollama die iGPU überhaupt anspricht

```
OLLAMA_FLASH_ATTENTION=1
```

zerlegt diese Aufgaben stattdessen in winzige Blöcke, sodass sie direkt im ultraschnellen Speicher des Grafikchips verarbeitet werden. Dadurch wird der ständige Zugriff auf den DDR4-RAM während der Berechnung verhindert, was die Verarbeitungsgeschwindigkeit erhöht und den Arbeitsspeicher spürbar

```
OLLAMA_KV_CACHE_TYPE=q4_0
```

Quantisiert den KV-Cache global für den Container – reduziert den Speicherbedarf um bis zu 72%

docker-compose.yml: Weitere Variablen

Diese Variablen wirken global für den gesamten Container.

Variable	Wert	Zweck
OLLAMA_IGPU_ENABLE	1	Schaltet die iGPU-Unterstützung im Container grundsätzlich frei
OLLAMA_NUM	1	NUMA-Awareness für Systeme mit mehreren Speicher-/CPU-Knoten
OLLAMA_NUM_PARALLEL	1	Begrenzt gleichzeitige Anfragen – kein paralleles Batching
OLLAMA_MAX_LOADED_MODELS	1	Nur ein Modell gleichzeitig geladen
OLLAMA_KEEP_ALIVE	-1	Hält das Modell dauerhaft im Speicher geladen – kein Neuladen bei jeder Anfrage, kostet aber permanent RAM/VRAM.
HSA_OVERRIDE_GFX_VERSION	11.0.0	Täuscht für Ollama eine offiziell unterstützte AMD-Grafikarchitektur vor, um die ROCm-Treiber zu überlisten (funktioniert mit spez. Grafikkarten).
GGML_VK_DISABLE_INTEGER_DOT_PRODUCT	1	Schaltet bestimmte mathematische Beschleunigungsbefehle (DP4A) im Vulkan-Treiber gezielt ab (Prozessor Bug).

Inhalt der docker-compose.yml

```
ollama:
  image: ollama/ollama:latest
  container_name: ollama_container
  ports:
    - "11434:11434"
  volumes:
    - ./ollama_data:/root/.ollama
  devices:
    - "/dev/dri"
    - "/dev/kfd"
  group_add:
    - "44"
    - "993"
  ipc: host
  ulimits:
    memlock:
      soft: -1
      hard: -1
  environment:
    - OLLAMA_VULKAN=1
    - OLLAMA_IGPU_ENABLE=1
    - OLLAMA_NUMA=1
    - OLLAMA_FLASH_ATTENTION=1
    - OLLAMA_KV_CACHE_TYPE=q4_0
    - OLLAMA_NUM_PARALLEL=1
    - OLLAMA_MAX_LOADED_MODELS=1
    - OLLAMA_KEEP_ALIVE=-1
    #- OLLAMA_CONTEXT_LENGTH=8192
  restart: unless-stopped
```

Modelfile: Basismodell, System-Rolle & Prompt-Format

FROM legt das Basismodell fest, SYSTEM definiert die permanente Rolle der KI und TEMPLATE bestimmt das exakte Prompt-Format – die Grundstruktur jeder Modelfile-Konfiguration.

```
$ cat Modelfile
FROM qwen2.5:14b

SYSTEM """
Du bist ein hochpräzises, sachliches
KI-Modell. Du extrahierst Fakten exakt
und halluzinierst nicht.
"""

TEMPLATE """
{{ if .System }}<|im_start|>system
{{ .System }}<|im_end|>{{ end }}
<|im_start|>assistant ...
"""
```

Die drei Bausteine der Modelfile

- 1 FROM – Basismodell (Pflichtfeld)
- 2 SYSTEM – permanente Rolle/Identität
- 3 TEMPLATE – exaktes Prompt-Format (Go-Syntax)

→ FROM ist Pflicht; SYSTEM und TEMPLATE sind optional, aber entscheidend für Tonalität und Formatstabilität. Zusätzliche MESSAGE-Paare liefern Few-Shot-Beispiele.

Modelfile: Parameter gegen Halluzinationen & Wiederholungen

Diese Parameter steuern, wie kreativ oder faktentreu das Modell antwortet – optimiert für Modell qwen2.5:14b.

Parameter (Halluzinationen)	Wert	Wirkung	Parameter (Wiederholungen)	Wert	Wirkung
temperature	0.7	Zufallsgenerator Niedrigere Werte machen die KI berechenbarer und fokussierter (Tools), höhere Werte experimentierfreudiger	presence_penalty	0.4	Themen-Wechsel Zwingt die KI, neue Begriffe und Konzepte einzubringen, statt auf alten Phrasen herumzureiten.
top_p	0.8	Themen-Filter Lässt die KI nur aus dem Pool der 80% wahrscheinlichsten Wörter wählen.	repeat_penalty	1.05	Wort-Sperre Bestraft direkte Wort- und Phrasenwiederholungen im generierten Text
top_k	20	Wortschatz-Bremse Begrenzt die Auswahl bei jedem Schritt strikt auf die 20 besten Wörter	repeat_last_n	64	Kurzzeitgedächtnis Legt fest, wie viele Wörter (Tokens) die KI rückwirkend im Blick behält, um Wiederholungen zu prüfen.
min_p	0.05	Qualitätsfilter Sortiert unlogische Wörter aus, die weniger als 5% Wahrscheinlichkeit haben.	Das Zusammenspiel: Größeres Gedächtnis (repeat_last_n) + stärkere Strafe (repeat_penalty) = maximaler Schutz vor Wiederholungen.		
			num_predict	-1	Not-Ausschalter Schneidet die Antwort nicht ab, sondern schreibt bis zum Ende.

Modelfile: Hardware & Speicher im Detail

Diese Parameter bestimmen den Arbeitsspeicher-Hebel und die Hardware-Zuteilung zwischen CPU und iGPU.

Parameter (Kontext & GPU-Zuteilung)	Wert	Wirkung	Parameter (CPU-Kerne)	Wert	Wirkung
num_ctx	8192	Setzt die Größe des Kontextfensters	num_thread	8	CPU-Kerne für Berechnungen, falls Arbeit auf die CPU ausgelagert wird.
num_gpu	-1	-1 verschiebt Schichten dynamisch auf die GPU (Auto-Erkennung), 0 = reine CPU, +1 = feste GPU-Schichten	seed	42	Fixiert den Zufallswert für identische Antworten beim Testen.

Modelfile: Stopp-Sequenzen & Few-Shot-Beispiele

Stop-Sequenzen erzwingen ein sauberes Ende der Antwort. MESSAGE-Paare geben dem Modell optional ein festes Beispielverhalten mit auf den Weg.

```
$ cat Modelfile
PARAMETER stop "<|im_start|>"
PARAMETER stop "<|im_end|>"
PARAMETER stop "\n\n\n"

MESSAGE user "Wie lautet die Hauptstadt
von Frankreich?"
MESSAGE assistant "Die Hauptstadt von
Frankreich ist Paris."
```

Stopp & Few-Shot auf einen Blick

- 1 Stop-Sequenzen – verhindern Selbstgespräche & Endlos-Output
- 2 Few-Shot – MESSAGE-Paare zeigen das gewünschte Antwortformat

→ Ohne Stop-Sequenzen kann das Modell über das Ziel hinausschreiben oder mit sich selbst weiterreden. Few-Shot-Beispiele sind optional, verbessern aber die Formattreue bei komplexen Aufgaben.

Erstellen des Modells

```
$ cp Modelfile ./ollama_data/
$ docker exec -it ollama_container ollama create mistral-opt -f \
/root/.ollama/Modelfile
```

GTT-Bereich manuell erweitern

Reicht der standardmäßige GTT-Bereich nicht aus, lässt sich die Obergrenze über einen Kernel-Parameter des amdgpu-Treibers anheben.

```
# Kernel-Parameter dauerhaft für amdgpu setzen
$ echo "options amdgpu gttsize=24000" | \
  sudo tee /etc/modprobe.d/amdgpu-gtt.conf

# Neustart, damit der Parameter geladen wird
$ reboot

# Neue GTT-Obergrenze prüfen (in MB)
$ echo "$(( $(cat /sys/class/drm/card0/device/mem_info_gtt_total) \
  / 1024 / 1024 )) MB"
24000 MB
```

24.000 MB

neue GTT-Obergrenze nach dem Neustart (≈ 23,4 GiB)

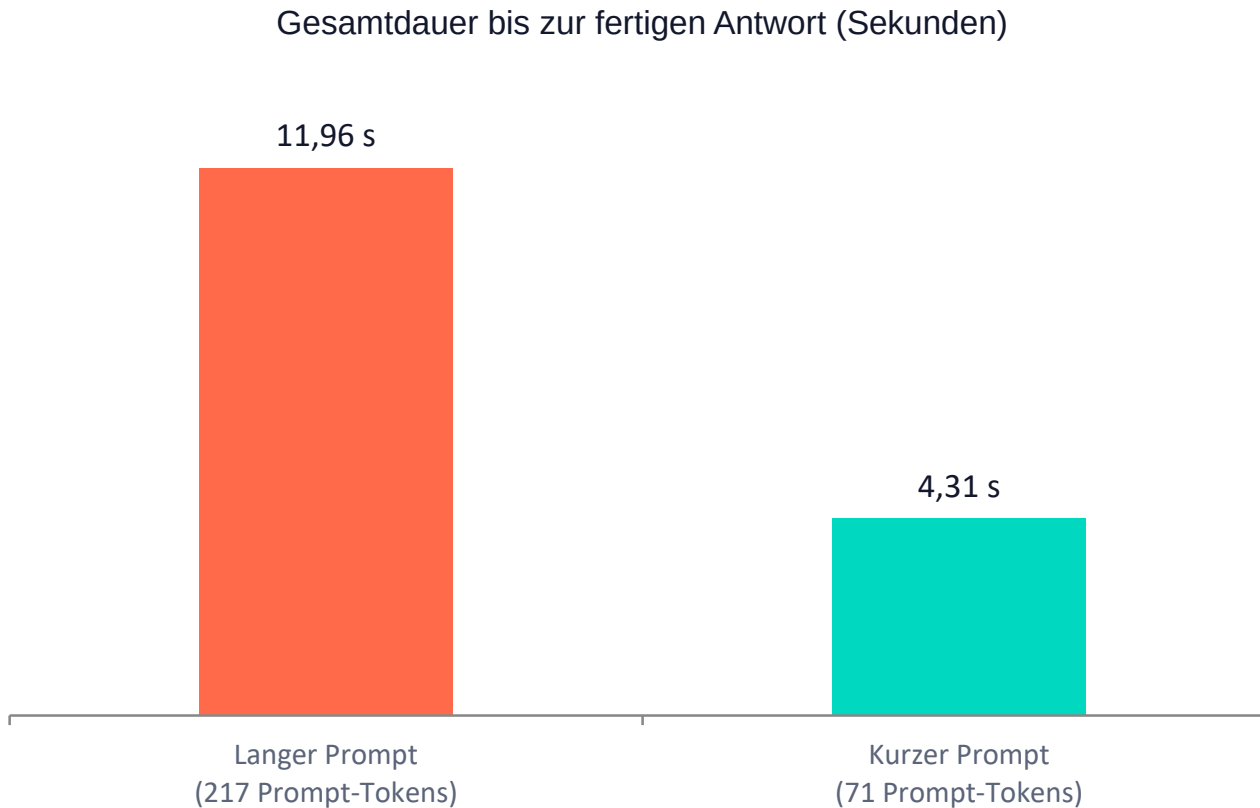
Zur Erinnerung (Kapitel 6)

GTT ist der RAM-Anteil, den sich die iGPU zusätzlich zum kleinen VRAM leihen darf. Der Kernel begrenzt diesen Bereich standardmäßig.

- gttsize wird in MB angegeben (hier: 24.000 MB)
- Der Wert sollte die BIOS-/UMA-Grenze nicht überschreiten
- Ein Neustart ist zwingend nötig – der Parameter wird beim Booten geladen

Prompt-Länge als versteckter Performance-Faktor

Gleiche Aufgabe, gleiches Modell: Ein aufgeblähter System-Prompt mit Anweisungen und Logs verlangsamt die Antwort erheblich – nicht die Antwortlänge ist entscheidend, sondern der Input.



~2,8x

schneller allein durch
Entfernen von unnötigem Prompt-Text

Praxis-Tipp

Lange Systemanweisungen, unnötige Kontrolleingaben und riesige Logs im Prompt kosten mehr Zeit als viele Server-Optimierungen zusammen.

Vielen Dank!

Fragen zu Ollama, Vulkan oder iGPU-Tuning?

